Práctica: Programación con Python

Documentación del Punto de Venta “Sweet Shop”

Alumno

Ian Hernández Ibarra

Carrera

Ingeniería en Software

Materia

Lenguajes de Programación

Fecha

27 de octubre de 2026

1. Introducción

Este documento explica el funcionamiento de “Sweet Shop”, un Punto de Venta (PdV) para una dulcería, desarrollado en Python con la biblioteca gráfica Tkinter. El programa permite iniciar sesión como cajero, seleccionar dulces con botones + y −, aplicar descuento a clientes VIP, calcular IVA, total y cambio, descontar existencias del inventario y generar un ticket que además se guarda en archivos de texto.

Cada sección describe una parte del programa, el código que la implementa y la captura de pantalla que muestra su comportamiento real.

2. Tecnologías y estructura general

El programa usa únicamente la biblioteca estándar de Python: `tkinter` para la interfaz y `messagebox` / `filedialog` para los cuadros de diálogo. Su código se organiza en el siguiente orden:

- **Datos del negocio:** diccionarios de productos, inventario y usuarios, más constantes de configuración.
- **Variables de control de Tkinter:** StringVar e IntVar que mantienen sincronizados los datos con la pantalla.
- **Funciones:** la lógica del sistema (cálculos, ventas, inventario, sesión).
- **Interfaz gráfica:** barra de menús, pantalla de login, marco de venta y panel del ticket, acomodados con Frame y grid.
- **Bucle principal:** root.mainloop() mantiene la ventana abierta esperando eventos.

3. Datos del negocio

3.1 Diccionarios

Los datos se guardan en tres diccionarios, donde la clave es un texto y el valor un número o una contraseña:

Diccionario	Contenido y uso
`PRODUCTOS`	Nombre del dulce → precio unitario (14 productos, por ejemplo Chocolates: $20, Gomitas: $15, Caramelos: $5). Se usa para generar la interfaz con un ciclo y calcular importes.
`INVENTARIO`	Nombre del dulce → piezas disponibles (por ejemplo Chocolates: 15, Brownies: 8). Se modifica cada vez que se confirma una venta o se reabastece.
`USUARIOS`	Usuario → PIN. Contiene admin (1234), cajero1 (0000) y cajero2 (1111). Se consulta en el inicio de sesión.

3.2 Constantes de configuración

Constante	Valor	Función
`IVA`	0.16	Porcentaje de impuesto aplicado después del descuento.
`STOCK_BAJO`	3	Si quedan 3 piezas o menos de un dulce, se muestra su stock en naranja y se avisa tras la venta.
`REABASTECER_CANTIDAD`	10	Piezas que se suman a cada dulce al reabastecer.
`PRODUCTOS_POR_COLUMNA`	7	Cantidad de productos por columna en la pantalla (se forman dos columnas).
`COLOR_*`	Tonos rojos	Paleta de colores de la interfaz (fondo, principal, suave y de sección).

3.3 Variables de control

Tkinter ofrece variables especiales que actualizan automáticamente los widgets enlazados. En el programa se usan, entre otras: `login_usuario` y `login_pin` (login), `usuario_activo` (cajero en sesión), `nombre_cliente`, `tipo_cliente` (1 = Regular, 2 = VIP), `numero_ticket` y `pago_cliente`. Además, por cada dulce se crean en un ciclo `vars_productos[nombre]` (cantidad elegida) y `vars_stock[nombre]` (texto de existencias). Los textos de subtotal, descuento, IVA, total, cambio y ticket también son StringVar.

4. Inicio de sesión

Al ejecutar el programa se abre una ventana pequeña (420×380) con el formulario de acceso construido en un Frame con grid: un campo de usuario, un campo de PIN con `show="*"` para ocultar los caracteres y el botón “Entrar”.

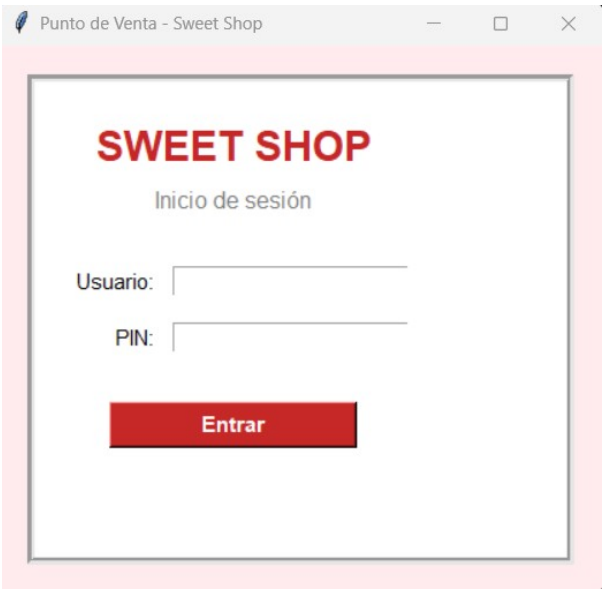


Figura 1. Pantalla de inicio de sesión.

Función iniciar_sesion()

Primero verifica que ambos campos tengan datos; si falta alguno muestra una advertencia. Después comprueba que el usuario exista en USUARIOS y que el PIN coincida:

- **Si son correctos:** guarda el usuario en usuario_activo, limpia el PIN, destruye el frame de login, agranda la ventana a 1180×780, activa la barra de menús y muestra el marco de venta y el panel del ticket.
- **Si son incorrectos:** borra el PIN y muestra el mensaje de error “Usuario o PIN incorrectos”.

```
if usuario in USUARIOS and USUARIOS[usuario] == pin:
    usuario_activo.set(usuario)
    frameLogin.destroy()
    root.geometry("1180x780")
    root.config(menu=barraMenu)
    miFrame.grid(row=0, column=0, ...)
    panelTicket.grid(row=0, column=1, ...)
else:
    messagebox.showerror("Acceso denegado", ...)
```

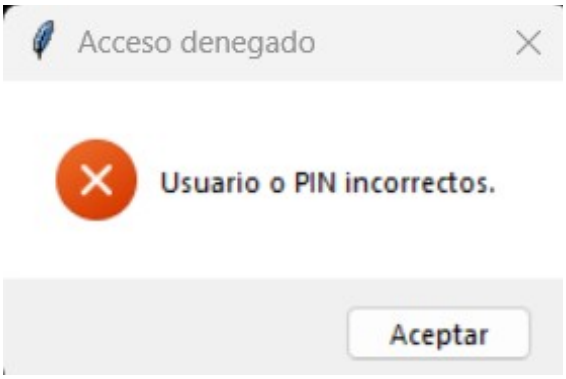


Figura 2. Mensaje de acceso denegado con credenciales incorrectas.

5. Pantalla principal de ventas

Con el acceso correcto aparece la pantalla principal, dividida en dos paneles: “NUEVA VENTA” a la izquierda y “TICKET” a la derecha. En la barra superior se ven los menús Archivo, Inventario y Ayuda.

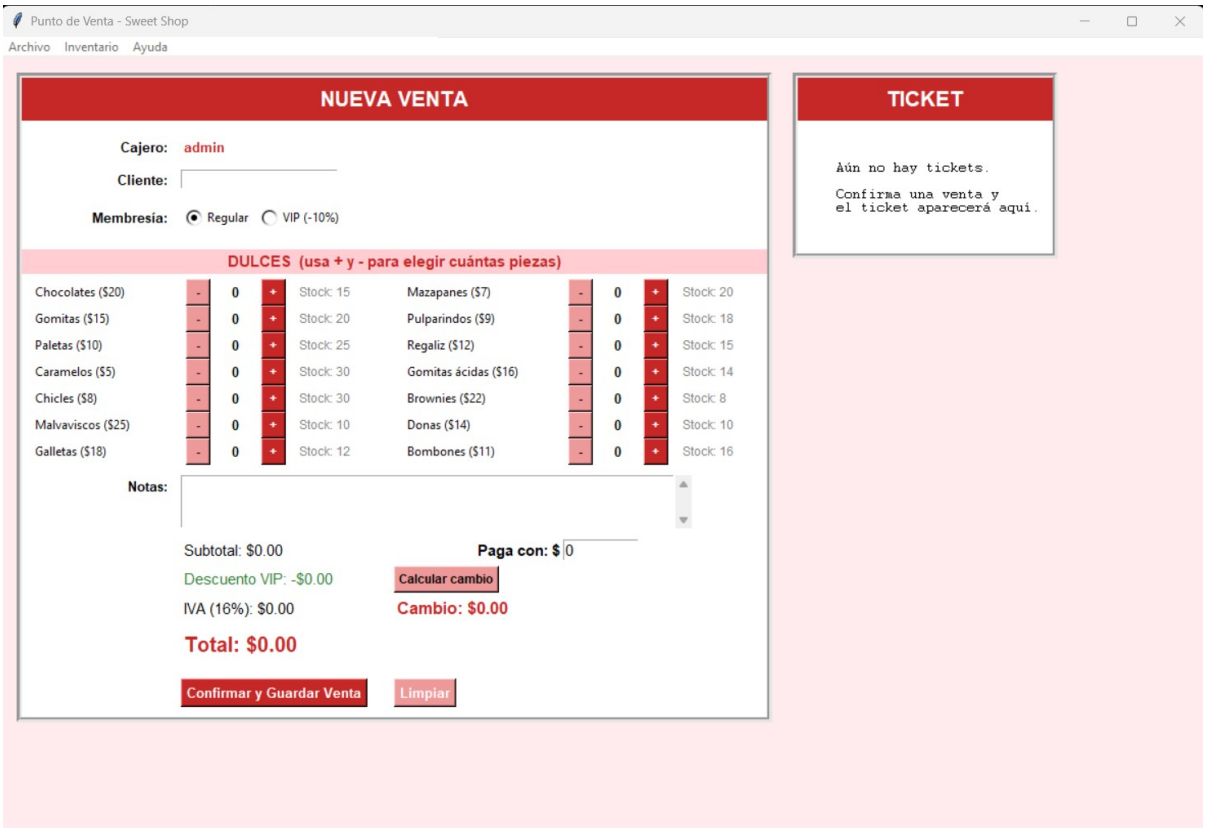


Figura 3. Pantalla principal del punto de venta con la sesión de admin.

Sus elementos son:

- **Cajero:** muestra el usuario que inició sesión (admin en la captura).
- **Cliente:** campo de texto obligatorio con el nombre del cliente.

- **Membresía:** dos Radiobutton, Regular o VIP (−10%). Al cambiarlos se recalcula el total automáticamente.
- **Dulces:** 14 filas generadas con un ciclo for sobre PRODUCTOS, en dos columnas de 7. Cada fila tiene nombre y precio, botón −, cantidad, botón + y existencias.
- **Notas:** widget Text con barra de desplazamiento (Scrollbar) para comentarios de la venta.
- **Resumen de cobro:** subtotal, descuento VIP, IVA y total, junto con el campo “Paga con”, el botón “Calcular cambio” y la etiqueta de cambio.
- **Botones:** “Confirmar y Guardar Venta” y “Limpiar”.
- **Panel TICKET:** muestra el último ticket generado. Al inicio dice “Aún no hay tickets”.

6. Funciones del programa

La siguiente tabla resume todas las funciones. Las más importantes se explican a detalle en las secciones siguientes.

Función	Qué hace
`actualizar_inventario()`	Refresca las etiquetas de stock y bloquea el botón + de los dulces agotados.
`calcular_subtotal()`	Suma precio × cantidad de todos los dulces.
`calcular_descuento()`	Regresa el 10% del subtotal si el cliente es VIP; si no, 0.
`calcular_iva()`	Calcula el 16% sobre el subtotal ya con descuento.
`calcular_total()`	Actualiza en pantalla subtotal, descuento, IVA y total; regresa el total.
`agregar_producto(nombre)`	Suma una pieza del dulce sin pasar de las existencias.
`quitar_producto(nombre)`	Resta una pieza (sin bajar de 0).
`calcular_cambio()`	Calcula el cambio o cuánto falta según lo que paga el cliente.
`limpiar_venta()`	Reinicia el formulario para una nueva venta.
`confirmar_venta()`	Valida, cobra, descuenta inventario, guarda archivos y muestra el ticket.
`ver_inventario()`	Muestra en un mensaje las existencias de todos los dulces.
`reabastecer_inventario()`	Suma 10 piezas a cada dulce previa confirmación.
`abrir_registro()`	Abre un explorador de archivos para localizar los .txt de registros.
`salir_app()`	Pregunta antes de cerrar el programa.
`iniciar_sesion()`	Valida usuario y PIN, y muestra el punto de venta.

6.1 Selección de productos: agregar_producto() y quitar_producto()

Cada botón + y − se crea dentro del ciclo con `lambda n=nombre: ...` para que cada botón recuerde a qué dulce pertenece. Al presionar +, la función compara la cantidad elegida contra el inventario; si ya se alcanzó el máximo, muestra la advertencia “Sin existencias”. En caso contrario aumenta la cantidad y llama a calcular_total() para refrescar los importes. El botón − funciona igual, pero solo resta si la cantidad es mayor a 0.

```
def agregar_producto(nombre):
    if vars_productos[nombre].get() >= INVENTARIO[nombre]:
        messagebox.showwarning("Sin existencias", ...)
        return
    vars_productos[nombre].set(vars_productos[nombre].get() + 1)
    calcular_total()
```

6.2 Cálculo de subtotal, descuento, IVA y total

Estas cuatro funciones trabajan en cadena. El orden de las operaciones es:

- **Subtotal =** suma de precio × cantidad de cada producto.
- **Descuento =** subtotal × 0.10, solo si tipo_cliente vale 2 (VIP).
- **IVA =** (subtotal − descuento) × 0.16.
- **Total =** subtotal − descuento + IVA.

calcular_total() además escribe los resultados en las etiquetas de la pantalla y reinicia el texto de cambio a $0.00.

6.3 Cálculo del cambio: calcular_cambio()

Lee el valor del campo “Paga con”. Si no es un número entero válido, muestra “Pago no válido (solo números)” (por eso `pago_cliente` es un IntVar). Si el pago es menor al total muestra “Faltan: $X”; si alcanza, muestra “Cambio: $X”.

6.4 Confirmación de la venta: confirmar_venta()

Es la función principal del programa. Sigue estos pasos en orden:

- **Recopila datos:** cajero, cliente, subtotal, descuento, IVA, total y notas (los saltos de línea se convierten en espacios).
- **Valida:** que haya nombre de cliente, que el total no sea 0 (venta vacía), que haya inventario suficiente y que el pago sea numérico y cubra el total.
- **Arma el ticket:** construye el texto con número de ticket (formato #0001), cajero, cliente y membresía, líneas de cada dulce, subtotal, descuento VIP (si aplica), IVA, total, pago y cambio.
- **Guarda en archivos:** escribe un registro en ventas.txt, otro en clientes.txt y la copia del ticket en tickets.txt, usando el modo "a" (añadir al final) para no borrar lo anterior.
- **Descuenta inventario:** resta de INVENTARIO las piezas vendidas.

- **Avisa stock bajo:** lista los dulces que quedaron con 3 piezas o menos.
- **Actualiza la pantalla:** muestra el ticket en el panel derecho, incrementa el número de ticket, limpia el formulario y muestra el mensaje de éxito con el cambio a entregar.

Todo el guardado está dentro de un bloque try/except, de modo que si ocurre un error al escribir los archivos se muestra un mensaje de error en lugar de cerrar el programa.

7. Ejemplo de una venta completa

Las siguientes capturas muestran una venta real a la clienta “Yurem” (membresía Regular) atendida por el cajero admin.

7.1 Captura de la compra y pago insuficiente

Se seleccionaron 3 Chocolates, 2 Galletas y 2 Regaliz. El programa calculó un subtotal de $120.00, IVA de $19.20 y un total de $139.20. El descuento VIP aparece en $0.00 porque la membresía es Regular. En el campo “Paga con” se escribió 20, que no alcanza para cubrir el total.



Figura 4. Venta en proceso: 3 Chocolates, 2 Galletas y 2 Regaliz.

Al presionar “Confirmar y Guardar Venta”, la validación del pago detectó que faltaban $119.20 y bloqueó la venta con una advertencia. No se descontó inventario ni se guardó ningún registro.



Figura 5. Advertencia de pago insuficiente.

7.2 Venta exitosa y ticket

Después se corrigió el pago a $200.00. Con ello la venta se confirmó, el cambio fue de $60.80 y el formulario se limpió para una nueva venta. El stock se actualizó: Chocolates pasó de 15 a 12, Galletas de 12 a 10 y Regaliz de 15 a 13. En el panel derecho apareció el ticket #0001 con el detalle completo de la compra.

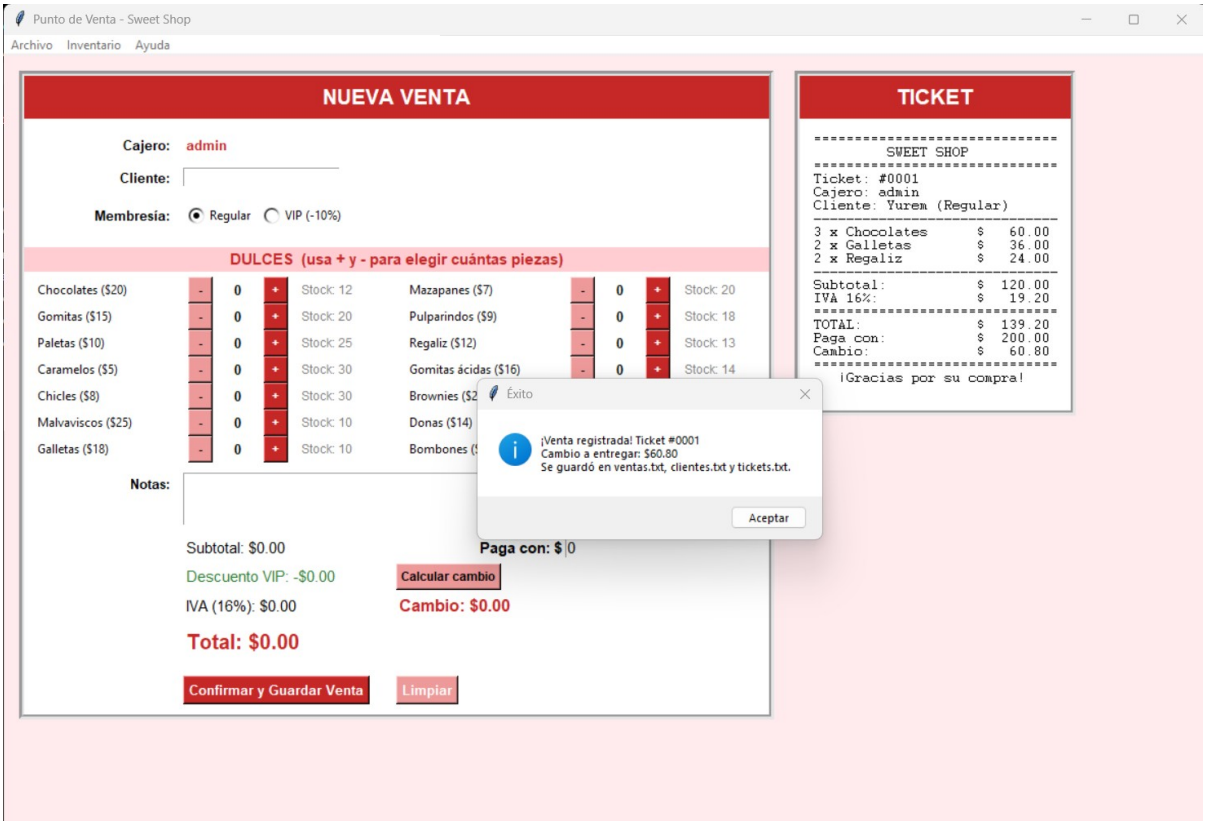


Figura 6. Venta registrada: mensaje de éxito, ticket generado e inventario actualizado.

La operación matemática de esta venta fue:

Concepto	Cálculo
`Chocolates`	3 × $20 = $60.00
`Galletas`	2 × $18 = $36.00

Concepto	Cálculo
`Regaliz`	2 × $12 = $24.00
`Subtotal`	$120.00
`Descuento VIP`	$0.00 (cliente Regular)
`IVA 16%`	$120.00 × 0.16 = $19.20
`Total`	$120.00 + $19.20 = $139.20
`Paga con / Cambio`	$200.00 − $139.20 = $60.80

8. Manejo del inventario

actualizar_inventario()

Se ejecuta al iniciar y después de cada venta o reabastecimiento. Recorre todos los productos y aplica tres estados visuales:

- **Normal (más de 3 piezas):** texto “Stock: N” en gris y botón + habilitado.
- **Stock bajo (3 piezas o menos):** texto en naranja, botón + habilitado.
- **Agotado (0 piezas):** texto “AGOTADO” en rojo, cantidad en 0 y botón + deshabilitado.

Menú Inventario

- **Ver inventario:** abre un mensaje con la lista de existencias de cada dulce.
- **Reabastecer todo:** después de confirmar, suma 10 piezas a cada producto y refresca la pantalla.

9. Barra de menús

Menú	Opciones
`Archivo`	Ver registros (abre un cuadro de diálogo de archivos con filedialog.askopenfilename y muestra la ruta del archivo elegido), Limpiar venta y Salir (con confirmación).
`Inventario`	Ver inventario y Reabastecer todo.
`Ayuda`	Acerca de: muestra el nombre y versión del programa (Software de Dulcería v4.0, creado con Tkinter).

La barra de menús se crea desde el inicio pero solo se activa con root.config(menu=barraMenu) después de un inicio de sesión exitoso, por lo que no aparece en la pantalla de login.

10. Archivos de registro

El programa genera tres archivos de texto en la misma carpeta del código, donde cada venta agrega una línea nueva:

Archivo	Contenido
`ventas.txt`	Ticket, cajero, cliente, membresía, dulces vendidos, subtotal, IVA, total, pago, cambio y notas.
`clientes.txt`	Cliente, membresía, ticket, compra, pago, cambio y cajero que lo atendió.
`tickets.txt`	Copia completa del ticket de cada venta.

La siguiente captura muestra clientes.txt abierto en Visual Studio Code junto a los demás archivos del proyecto. Se observa el registro de la venta de Yurem en la última línea ($139.20, pagó $200.00, cambio $60.80, atendió admin).

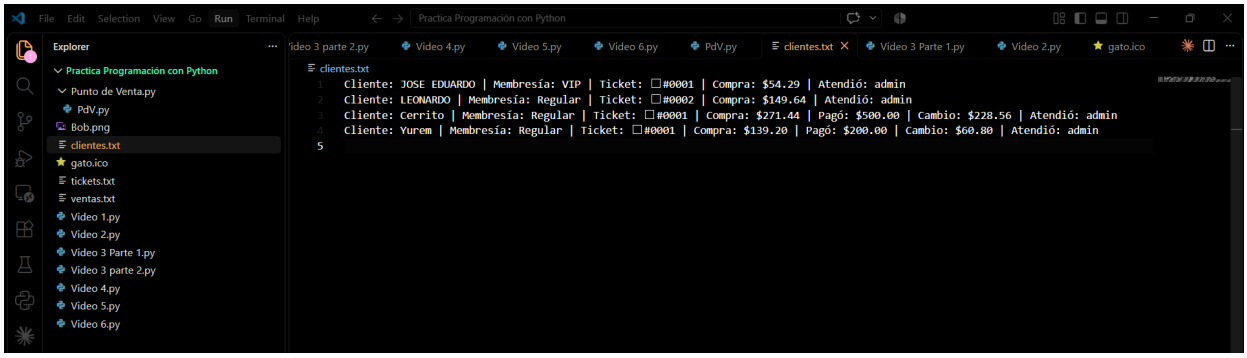


Figura 7. Archivo clientes.txt con los registros de clientes en Visual Studio Code.

Observación: en el archivo se ve que varios clientes tienen el ticket #0001. Esto ocurre porque numero_ticket vuelve a valer 1 cada vez que se abre el programa. Una mejora posible sería leer el último número guardado en tickets.txt al iniciar para continuar la numeración.

11. Conclusión

El punto de venta Sweet Shop integra varios conceptos de programación en Python: diccionarios para los datos, funciones para separar la lógica, ciclos para generar la interfaz, condicionales para las validaciones, manejo de excepciones con try/except, manejo de archivos de texto y una interfaz gráfica completa con Tkinter (Frame, grid, Entry, Radiobutton, Text, Scrollbar, Menu y cuadros de diálogo). Las capturas confirman que el sistema valida el acceso, calcula correctamente los importes, controla el inventario y conserva el historial de ventas.